# SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

**DOCUMENT 5 OF 10**

## Databases — PostgreSQL, Indexing & Transactions

*10 Questions | MVCC | ACID | N+1 | Deadlocks | EXPLAIN ANALYZE | Partitioning*

# Databases — PostgreSQL, Indexing & Transactions

## Q1: How do indexes work internally — B-tree structure and tradeoffs?

An index is a separate data structure maintained by the database that allows fast lookups without a full table scan. Understanding the B-tree structure explains why certain queries use indexes and others don't.

**B-tree structure:** A balanced tree where each node holds multiple keys and pointers. Leaf nodes contain the actual indexed values and pointers to the table rows (heap tuples in PostgreSQL). Inner nodes are navigation guides. The tree is kept balanced — insert/delete rebalance automatically. Height is O(log N) — for a table with a billion rows, a B-tree lookup touches ~30 nodes.

**Why range queries work:** B-tree nodes are sorted. A range query (WHERE age BETWEEN 20 AND 30) finds the start of the range in O(log N) then scans leaf nodes sequentially — no random IO for the range portion.

**Composite index column order:** A composite index on (last_name, first_name) supports: WHERE last_name = 'X', WHERE last_name = 'X' AND first_name = 'Y'. It does NOT efficiently support WHERE first_name = 'Y' alone — the leftmost prefix rule. Always put the most selective and most queried column first.

```
-- PostgreSQL index types
CREATE INDEX idx_users_email ON users(email);          -- B-tree default
CREATE INDEX idx_users_name  ON users(last_name, first_name);  -- composite
CREATE INDEX idx_lower_email ON users(LOWER(email));   -- expression index
CREATE INDEX idx_active      ON users(id) WHERE active = true; -- partial index

-- EXPLAIN ANALYZE to verify index usage
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'g@test.com';
-- Index Scan using idx_users_email on users (cost=0.43..8.45 rows=1)
-- vs Seq Scan (cost=0.00..2841.00 rows=100000) -- 350x slower

-- Partial index: much smaller, faster for common query pattern
-- Only indexes rows where active=true instead of all rows
CREATE INDEX idx_active_users ON users(created_at) WHERE active = true;
```

**Index bloat:** Frequent UPDATEs and DELETEs cause dead tuples in PostgreSQL. Run VACUUM ANALYZE regularly. REINDEX to rebuild bloated indexes. Monitor pg_stat_user_indexes for index usage — unused indexes waste write performance and disk space.

---

## Q2: ACID — what each property means and how PostgreSQL implements them.

ACID is the set of guarantees a transactional database provides. Knowing the implementation details lets you reason about what can and cannot go wrong.

**Atomicity:** All operations in a transaction succeed or all are rolled back. No partial commits. PostgreSQL implements this via WAL (Write-Ahead Log) — changes are written to the log before the data files. On crash, incomplete transactions are rolled back from the WAL during recovery.

**Consistency:** The database moves from one valid state to another. Constraints (NOT NULL, UNIQUE, FK, CHECK) are enforced at commit time. If any constraint is violated, the whole transaction rolls back.

**Isolation:** Concurrent transactions don't interfere with each other. Implemented via MVCC (Multi-Version Concurrency Control) in PostgreSQL — readers never block writers, writers never block readers. Each transaction sees a snapshot of the DB at its start time.

**Durability:** Committed transactions survive crashes. PostgreSQL's WAL ensures this — fsync() writes the log to disk before acknowledging a commit. If the process crashes immediately after commit, the WAL replays on startup.

```
-- Explicit transaction
BEGIN;
```

```
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- Both updates happen, or neither does (atomicity)
COMMIT;

-- ROLLBACK on failure
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  -- some error occurs
ROLLBACK;  -- first update is undone

-- Savepoints for partial rollback
BEGIN;
  INSERT INTO log VALUES ('step 1');
  SAVEPOINT step1;
  INSERT INTO log VALUES ('step 2');
  ROLLBACK TO SAVEPOINT step1;  -- undo only step 2
COMMIT;  -- only step 1 committed
```

---

## Q3: Transaction isolation levels — what anomalies each level prevents.

Isolation levels are a tradeoff between correctness and concurrency. Higher isolation = fewer anomalies = more locking = lower throughput. PostgreSQL's default is READ COMMITTED; serializable is the strongest.

**Read Uncommitted:** Can read uncommitted changes from other transactions (dirty reads). PostgreSQL doesn't actually implement this — it falls back to READ COMMITTED. Basically never use this.

**Read Committed (PostgreSQL default):** Each statement sees data committed before that statement started. Prevents dirty reads. Allows non-repeatable reads (same SELECT in same transaction can return different rows if another transaction commits between them) and phantom reads.

**Repeatable Read:** Transaction snapshot taken at transaction start. All reads see the same data. Prevents dirty reads and non-repeatable reads. Still allows phantoms in standard SQL (PostgreSQL actually prevents phantoms too via MVCC — stronger than the spec requires).

**Serializable:** Transactions appear to execute one at a time. Prevents all anomalies. PostgreSQL uses Serializable Snapshot Isolation (SSI) — detects dangerous dependency cycles and aborts one transaction. Use for financial calculations, inventory management, anything requiring perfect consistency.

```
-- Non-repeatable read example (READ COMMITTED)
-- T1:                                T2:
SELECT balance FROM accounts        -- balance = 100
WHERE id = 1;   -- returns 100
                                    UPDATE accounts SET balance = 200
                                    WHERE id = 1;
                                    COMMIT;
SELECT balance FROM accounts
WHERE id = 1;   -- returns 200!    -- Different result in same transaction

-- Setting isolation level
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 1;  -- 100
  -- T2 commits and changes to 200
  SELECT balance FROM accounts WHERE id = 1;  -- still 100 (snapshot)
COMMIT;

-- Serializable for critical sections
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

---

## Q4: The N+1 query problem — how to detect and fix it.

The N+1 problem is one of the most common performance killers in applications using ORMs. It occurs when code fetches a list of N items, then makes an additional query for each item — resulting in N+1 total queries instead of 1 or 2.

```
# N+1 example with SQLAlchemy (the silent killer)
users = db.query(User).limit(100).all()   # 1 query
for user in users:
    print(user.orders)   # 1 query per user = 100 queries!
# Total: 101 queries for what should be 1-2 queries

# Fix 1: Eager loading with joinedload
users = db.query(User).options(joinedload(User.orders)).limit(100).all()
# Generates: SELECT users.*, orders.* FROM users
#            LEFT JOIN orders ON orders.user_id = users.id
# Total: 1 query

# Fix 2: subqueryload (separate query but batched)
users = db.query(User).options(subqueryload(User.orders)).limit(100).all()
# Query 1: SELECT * FROM users
# Query 2: SELECT * FROM orders WHERE user_id IN (1,2,...,100)
# Total: 2 queries — better for large result sets

# Fix 3: Raw SQL with JOIN (most control)
result = db.execute('''
    SELECT u.*, o.id as order_id, o.total
    FROM users u
    LEFT JOIN orders o ON o.user_id = u.id
    WHERE u.active = true
''')
```

**Detection:** Enable SQL logging in development (SQLAlchemy echo=True, Django DEBUG=True). Count queries per request. django-debug-toolbar, SQLAlchemy-query-tracker. In production, use slow query logs and APM tools (Datadog, New Relic) to find endpoints making excessive queries.

**GraphQL N+1:** GraphQL resolvers are particularly prone to N+1. Solution: DataLoader pattern — batch all IDs collected during a request, fetch them in one query, distribute results. Facebook's dataloader library implements this.

---

## Q5: Deadlocks — how they happen, how to detect, and how to prevent them.

A deadlock occurs when two or more transactions each hold a lock that the other needs, creating a circular wait. PostgreSQL detects deadlocks automatically and kills one transaction (the deadlock victim).

```
-- Classic deadlock scenario
-- Transaction A:                  Transaction B:
BEGIN;                             BEGIN;
UPDATE accounts                    UPDATE accounts
  SET balance = balance - 100        SET balance = balance + 50
  WHERE id = 1;   -- locks row 1     WHERE id = 2;  -- locks row 2

UPDATE accounts                    UPDATE accounts
  SET balance = balance + 100        SET balance = balance - 50
  WHERE id = 2;   -- WAITS for B     WHERE id = 1; -- WAITS for A

-- PostgreSQL detects cycle, kills one transaction:
-- ERROR: deadlock detected
-- DETAIL: Process 12345 waits for ShareLock on transaction 67890
```

```
-- Prevention: always acquire locks in the same order
-- Both transactions should lock row 1 THEN row 2
BEGIN;
SELECT * FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE;
-- Locks both rows in consistent order before any updates
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

**Prevention strategies:** (1) Always acquire multiple locks in the same consistent order (sort by ID). (2) Keep transactions short — the shorter the transaction, the less time locks are held. (3) Use SELECT ... FOR UPDATE to acquire row locks explicitly before modifying. (4) Use NOWAIT or SKIP LOCKED to fail fast rather than wait.

**Application-level retry:** Deadlocks are inevitable in high-concurrency systems. Always wrap transactions in retry logic that catches deadlock errors (PostgreSQL error code 40P01) and retries with exponential backoff.

---

## Q6: How does MVCC work in PostgreSQL?

MVCC (Multi-Version Concurrency Control) is PostgreSQL's mechanism for providing isolation without reader-writer blocking. Instead of locking rows for reads, it keeps multiple versions of each row.

**Row versioning:** Every row in PostgreSQL has hidden system columns: xmin (transaction ID that created this version) and xmax (transaction ID that deleted/updated this version, or 0 if still live). When you UPDATE a row, PostgreSQL doesn't modify it — it inserts a new version and marks the old one with xmax.

**Snapshot isolation:** When a transaction starts, it records the current transaction ID as its snapshot. It can only see rows where xmin <= snapshot AND (xmax = 0 OR xmax > snapshot). This means each transaction sees a consistent point-in-time snapshot regardless of concurrent writes.

**Why reads don't block writes:** Readers read from their snapshot — they never wait for writers to finish. Writers create new row versions — they never wait for readers. Only writer-writer conflicts cause blocking (two transactions trying to update the same row).

**Dead tuples and VACUUM:** Old row versions accumulate as dead tuples. VACUUM reclaims this space. autovacuum runs automatically. Without VACUUM, tables grow unboundedly and table bloat slows queries. monitor pg_stat_user_tables for n_dead_tup.

```
-- See MVCC in action
-- Session 1:
BEGIN;
SELECT txid_current();   -- e.g., 1000
SELECT * FROM users WHERE id = 1;  -- sees xmin <= 1000

-- Session 2 (concurrent):
UPDATE users SET name = 'New Name' WHERE id = 1;
COMMIT;   -- creates new row version with xmin=1001

-- Session 1 (still in transaction):
SELECT * FROM users WHERE id = 1;  -- still sees OLD version (xmin=999)
-- Session 1 snapshot is 1000, new version has xmin=1001 > 1000
COMMIT;
```

---

## Q7: Query optimization — how to read EXPLAIN ANALYZE and fix slow queries.

EXPLAIN ANALYZE is the most powerful tool for understanding why a query is slow. It shows the execution plan the query planner chose, and the actual runtime statistics.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.name, COUNT(o.id) as order_count
FROM users u
```

```
LEFT JOIN orders o ON o.user_id = u.id
WHERE u.created_at > '2024-01-01'
GROUP BY u.id, u.name
ORDER BY order_count DESC
LIMIT 10;

-- Sample output to read:
-- Limit  (cost=842.51..842.53 rows=10 width=44) (actual time=12.3..12.3 rows=10)
--   -> Sort  (cost=842.51..867.51 rows=10000) (actual time=12.3..12.3 rows=10)
--       -> HashAggregate (cost=... actual rows=10000 loops=1)
--           -> Hash Left Join (cost=... actual rows=50000 loops=1)
--               Hash Cond: (o.user_id = u.id)
--               -> Seq Scan on orders (rows=500000)   <- PROBLEM
--               -> Hash -> Index Scan on users

-- Seq Scan on a large table = missing index
-- Fix:
CREATE INDEX idx_orders_user_id ON orders(user_id);
-- Also add for the filter:
CREATE INDEX idx_users_created_at ON users(created_at);
```

**Key nodes to spot:** Seq Scan on large tables = missing index. Nested Loop with large row counts = join order problem or missing index. Hash Join = good for large joins. Sort without index = consider index for ORDER BY column. Bitmap Heap Scan = good, recheck condition = index isn't covering.

**Covering indexes:** If a query only needs columns that are all in the index, PostgreSQL can answer it from the index alone (Index Only Scan) — never touching the heap. Dramatically faster. CREATE INDEX idx_cover ON users(email) INCLUDE (name, id);

**pg_stat_statements:** Enable this extension to track query performance over time — total calls, total time, mean time per execution. The #1 tool for finding production slow queries without modifying application code.

---

## Q8: Connection pooling — why it matters and how PgBouncer works.

Opening a PostgreSQL connection is expensive — it forks a process (~5MB RAM), performs authentication, and negotiates SSL. For APIs handling hundreds of concurrent requests, connecting per-request would be catastrophic.

**Application-level pooling:** SQLAlchemy, asyncpg, psycopg3 all maintain connection pools within the application process. Pool size = min to keep warm, max to allow. Requests wait if all connections are busy (pool timeout).

**PgBouncer:** A dedicated connection pooler that sits between your app and PostgreSQL. Multiple app servers share a small number of actual PostgreSQL connections. 10,000 app connections → 100 real DB connections. Three modes: session pooling (connection held for session duration), transaction pooling (connection returned to pool after each transaction — most efficient), statement pooling (after each statement — breaks multi-statement transactions, rarely used).

```
# Without pooler:
# 10 app servers × 20 connections each = 200 PostgreSQL processes
# Each process: ~5MB RAM = 1GB just for connections

# With PgBouncer (transaction pooling):
# 10 servers × 20 connections → PgBouncer → 20 real PG connections
# 10x reduction in PostgreSQL memory usage

# PgBouncer config (pgbouncer.ini)
[databases]
mydb = host=localhost port=5432 dbname=mydb

[pgbouncer]
pool_mode = transaction
max_client_conn = 1000    # app connections PgBouncer accepts
default_pool_size = 20    # real PostgreSQL connections per DB
```

```
min_pool_size = 5
reserve_pool_size = 5     # extra connections for bursts
server_idle_timeout = 600
```

**Transaction pooling gotcha:** With transaction pooling, SET LOCAL, advisory locks, and prepared statements don't work correctly — the connection is returned to the pool after the transaction, so state set on the connection is lost. Ensure your ORM doesn't use these features in ways that assume connection persistence.

---

## Q9: Database normalization and when to deliberately denormalize.

Normalization reduces data redundancy and maintains integrity. Denormalization trades integrity for read performance. Both are tools — the right choice depends on read/write ratio and query patterns.

**1NF:** No repeating groups, atomic values. Each cell contains a single value. No comma-separated lists in a column.

**2NF:** 1NF + no partial dependencies. Every non-key column depends on the FULL primary key, not just part of it. Relevant when you have composite primary keys.

**3NF:** 2NF + no transitive dependencies. Non-key columns should not depend on other non-key columns. e.g., storing city AND zip code together — zip determines city, so city is transitively dependent.

```
-- Normalized (3NF): orders and products separate
orders(id, user_id, created_at)
order_items(order_id, product_id, quantity, price_at_purchase)
products(id, name, current_price)

-- Query: get all items for order 42 with product names
SELECT oi.quantity, oi.price_at_purchase, p.name
FROM order_items oi
JOIN products p ON p.id = oi.product_id
WHERE oi.order_id = 42;

-- Denormalized for reporting (analytics DB / OLAP)
order_facts(
  order_id, user_id, user_email, user_name,   -- duplicated user data
  product_id, product_name, product_category,  -- duplicated product data
  quantity, price, revenue
)
-- No joins needed for dashboards — fast aggregations
-- Data updated by ETL pipeline, not in real-time
```

**When to denormalize:** Read-heavy analytics and reporting queries. Materialized views as a middle ground — normalized source, denormalized view refreshed on schedule. Event sourcing systems where the event log is the truth and projections are derived. Caching frequently joined data in Redis.

---

## Q10: Partitioning strategies in PostgreSQL — when and how.

Table partitioning splits a large table into smaller physical chunks while presenting a single logical table to queries. PostgreSQL's declarative partitioning (v10+) makes this straightforward.

**Range partitioning:** Partition by a range of values — most common for time-series data. Partition by month/year. New partitions created as time advances. Old partitions can be archived or dropped cheaply (DROP TABLE on the partition, not DELETE).

**List partitioning:** Partition by a list of discrete values — e.g., region (US, EU, APAC) or status. Good when you frequently query by that dimension and want partition pruning.

**Hash partitioning:** Distribute rows evenly across N partitions based on hash of a key. Useful for sharding by user_id or similar. Ensures even distribution without temporal skew.

```
-- Range partitioning by month
CREATE TABLE events (
    id         BIGSERIAL,
    created_at TIMESTAMPTZ NOT NULL,
    payload    JSONB
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2024_01
    PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE events_2024_02
    PARTITION OF events
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Query planner automatically prunes partitions:
EXPLAIN SELECT * FROM events WHERE created_at >= '2024-01-01'
                               AND created_at  < '2024-02-01';
-- -> Seq Scan on events_2024_01 (only scans this partition)

-- Drop old data cheaply (no DELETE, no VACUUM needed)
DROP TABLE events_2023_01;   -- instant, no transaction log bloat
```

**Partition pruning:** The query planner eliminates partitions that cannot satisfy the WHERE clause. For this to work, the partition key must appear in the WHERE clause. Always use the partition key in your most common query filters.

---